Ordenação: Fundamentos e Comparação

Resumo completo — baseado em `Aula6_AnaliseOrdenacao.pdf` (partes gerais) e `Lista6_AnaliseOrdenacao.pdf` (Exercícios 1, 2, 3, 7, 8 e 14).

Cada algoritmo tem o seu próprio resumo: Bubble · Selection · Insertion · Merge · Quick · Heap. A base matemática (recorrências, somas, logaritmos) está em 01 — Recursividade e Recorrências.

---

1. Por que estudar ordenação?

Ordenar é reorganizar os elementos segundo um critério (crescente, decrescente...).

> **Pergunta:** se dois algoritmos produzem a mesma sequência ordenada, eles são igualmente eficientes?
> **Não.** Podem diferir em: número de comparações, quantidade de movimentações, tempo de execução, memória auxiliar e desempenho em entradas já ordenadas ou invertidas.

2. O que é ordenar corretamente?

Entrada `[8, 3, 5, 3, 1]` → saída crescente válida `[1, 3, 3, 5, 8]`.

Dois requisitos:

1. **Ordem:** os elementos respeitam o critério.
2. **Preservação dos elementos:** nada é perdido, criado ou duplicado. A saída é uma **permutação** da entrada.

Ordenar **não** é eliminar duplicatas. Se o algoritmo devolver `[1, 3, 5, 8]`, a sequência está em ordem crescente, mas **não** é uma ordenação correta da entrada (perdeu um 3).

3. Critérios de análise

Critério	Pergunta
Tempo	Quanto cresce o trabalho com o tamanho da entrada?
Melhor caso	O que acontece quando a entrada favorece o algoritmo?
Pior caso	Qual é o maior custo para entradas de tamanho n?
Caso médio	Qual é o custo esperado segundo um **modelo de entradas**?
Memória	Quanto espaço auxiliar é usado?
Estabilidade	A ordem relativa de elementos iguais é preservada?

Os critérios são **independentes**: um algoritmo pode ser rápido e ainda assim precisar de memória extra ou não ser estável.

> **Caso médio ≠ ponto médio** entre melhor e pior caso. Ele depende de um **modelo de probabilidade** das entradas (por exemplo, "todas as permutações são igualmente prováveis"), e as hipóteses devem ser explicitadas.

Exemplo com Insertion Sort:

Entrada	Situação	Custo
`1 2 3 4 5`	melhor caso — já ordenado	poucos deslocamentos: $\Theta(n)$
`2 4 1 5 3`	intermediário — parcialmente desordenado	depende de quantos elementos precisam ser deslocados
`5 4 3 2 1`	pior caso — ordem inversa	muitos deslocamentos: $\Theta(n^2)$

4. In-place

Um algoritmo **in-place** ordena diretamente na estrutura original, usando pouca memória auxiliar (normalmente $O(1)$).

- **In-place:** Bubble, Selection, Insertion e Heap Sort. Quick Sort também, nas implementações usuais, embora use **pilha de recursão**.
- **Não in-place:** Merge Sort tradicional para vetores (vetor auxiliar proporcional a n).

> **Cuidado conceitual:** in-place **não** significa "nenhuma memória adicional". É preciso distinguir **espaço auxiliar para dados** de **espaço da pilha de recursão** (veja a seção 9 de Recursividade).

5. Estabilidade

Um algoritmo é **estável** quando preserva a ordem relativa dos elementos com a **mesma chave**.

Exemplo — alunos ordenados por nota:

Entrada	Ana (8)	Bruno (6)	Caio (8)
Saída estável	Bruno (6)	Ana (8)	Caio (8)

Ana continua antes de Caio: ambos têm nota 8 e Ana aparecia primeiro.

Por que importa? Registros reais têm outras informações além da chave (funcionários por salário, produtos por preço, alunos por nota). Se os dados já estavam ordenados por outro critério (por exemplo, nome), um algoritmo estável **preserva** essa ordem anterior entre os empates. Por isso, ao reordenar uma lista que já foi ordenada por outro critério, a propriedade a priorizar é a **estabilidade**.

Estabilidade não é o mesmo que in-place

Algoritmo	In-place?	Estável?
Bubble Sort	Sim	Sim, na versão usual (troca só com `>`)
Selection Sort	Sim	Não, na versão usual (a troca "salta" elementos)
Insertion Sort	Sim	Sim (desloca só com `>`)
Merge Sort	Geralmente não (vetor auxiliar)	Sim, se implementado corretamente (`<=` na intercalação)
Quick Sort	Geralmente sim	Não, na versão usual
Heap Sort	Sim	Não

Essas classificações dependem da implementação: há Merge Sort com estratégias especiais de memória, e um Quick Sort estável exige implementação diferente da usual.

6. Conceito útil: inversões

Uma **inversão** é um par de posições $i < j$ com $v[i] > v[j]$ (par "fora de ordem").

- Vetor ordenado: **0** inversões. Vetor invertido: o máximo, $\binom{n}{2} = \dfrac{n(n-1)}{2}$.
- Em média (permutações aleatórias), cada par está invertido com probabilidade $1/2$: $\dfrac{n(n-1)}{4}$ inversões.
- Cada **troca do Bubble Sort** (vizinhos) e cada **deslocamento do Insertion Sort** corrige **exatamente uma** inversão. Logo, nesses dois algoritmos, nº de trocas/deslocamentos = nº de inversões. Isso explica por que eles são $\Theta(n^2)$ no pior e no caso médio.

7. Por que precisamos de algoritmos mais eficientes?

Comparação ilustrativa das funções (não são tempos medidos):

n	n^2	$n \log_2 n$	**razão**
10	100	≈ 33	3×
100	10.000	≈ 664	15×
1.000	1.000.000	≈ 9.966	100×
10.000	100.000.000	≈ 132.877	750×
1.000.000	10^{12}	$\approx 2 \times 10^7$	50.000×

Mensagem central: para entradas grandes, a diferença entre n^2 e $n \log n$ é enorme. Na prática, constantes, implementação e características dos dados também influenciam.

8. Tabelas comparativas

Tempo (implementações convencionais)

Algoritmo	Melhor caso	Caso médio	Pior caso
Bubble Sort (sem parada antecipada)	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$
Bubble Sort otimizado (com parada)	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$
Selection Sort	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$
Insertion Sort	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$
Merge Sort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$
Quick Sort clássico	$\Theta(n \log n)$	$\Theta(n \log n)$ *	$\Theta(n^2)$
Heap Sort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$

* sob condições usuais sobre a entrada e a escolha do pivô.

Memória e estabilidade

Algoritmo	Espaço auxiliar típico	In-place?	Estável?
Bubble Sort	$\Theta(1)$	Sim	Sim
Selection Sort	$\Theta(1)$	Sim	Não
Insertion Sort	$\Theta(1)$	Sim	Sim
Merge Sort	$\Theta(n)$	Não, na versão tradicional	Sim
Quick Sort	pilha de recursão: média $\Theta(\log n)$, pior $\Theta(n)$	Geralmente sim	Não
Heap Sort	$\Theta(1)$ (heapify iterativo)	Sim	Não

> **Ponto de atenção:** a tabela não é uma propriedade imutável de qualquer código com o nome do algoritmo. Implementações específicas podem alterar estabilidade, memória e comportamento. **Analise o código concreto.**

Estratégias, em uma linha

Algoritmo	Estratégia
Bubble	compara vizinhos e troca quando necessário; o maior "borbulha" para o fim
Selection	seleciona o menor da parte não ordenada e o coloca na próxima posição
Insertion	insere cada elemento na posição correta da região já ordenada
Merge	divide em metades, ordena recursivamente e **intercala**
Quick	**particiona** em torno de um pivô e ordena recursivamente os lados (sem intercalação)
Heap	constrói um heap máximo e extrai repetidamente o maior elemento

9. Como escolher um algoritmo?

Não existe algoritmo universalmente melhor.

Situação	Escolha	Motivo
Entrada pequena ou quase ordenada	**Insertion Sort**	simples, $\Theta(n)$ no melhor caso, estável, in-place
Garantia de tempo no pior caso	**Merge Sort** ou **Heap Sort**	$\Theta(n \log n)$ no pior caso
Preservar a ordem de registros com chaves iguais	**Merge Sort** (ou Insertion, se pequeno)	estáveis
Pouca memória auxiliar + garantia $n \log n$	**Heap Sort**	in-place, $\Theta(n \log n)$ sempre

Considere também a linguagem, a biblioteca, a distribuição dos dados e o custo de comparar × movimentar elementos.

---

10. Exercícios resolvidos (Lista 6)

Exercício 1 — primeira passagem em `[7, 3, 5, 2]`

Bubble Sort (compara vizinhos):

Comparação	Resultado	Vetor
7 e 3	troca	`[3, 7, 5, 2]`
7 e 5	troca	`[3, 5, 7, 2]`
7 e 2	troca	`[3, 5, 2, 7]`

3 comparações, 3 trocas. O 7 "borbulhou" até o fim. Vetor após a passagem: **`[3, 5, 2, 7]`**.

Selection Sort (procura o menor de `v[0..3]`):

Comparação	Menor até agora
início	7 (pos 0)
3 < 7? sim	3 (pos 1)
5 < 3? não	3 (pos 1)
2 < 3? sim	2 (pos 3)

3 comparações; troca `v[0]` ↔ `v[3]` (mudam de posição o 7 e o 2). Vetor: **`[2, 3, 5, 7]`** (por coincidência já ordenado, mas o algoritmo ainda fará as outras passagens).

Insertion Sort (primeira iteração, `chave = 3`):

Passo	Ação	Vetor
7 > 3? sim	desloca 7 para a direita	`[7, 7, 5, 2]`
`j < 0`	para; insere 3 na posição 0	`[3, 7, 5, 2]`

1 comparação entre elementos. Mudam de posição o 7 (deslocado) e o 3 (inserido). Vetor: `[3, 7, 5, 2]`.

Qual faz seleção explícita do menor? O **Selection Sort**.

Exercício 2 — associe a propriedade

1. Ordenar funcionários por salário preservando a ordem anterior entre salários iguais → **Estabilidade**.
2. Ordenar sem vetor auxiliar proporcional a n → **In-place**.
3. Garantir $\Theta(n \log n)$ mesmo no pior caso → **Heap Sort ou Merge Sort**.

Exercício 3 — pior caso $\Theta(n \log n)$, pouca memória, estabilidade não obrigatória

a) **Heap Sort**. b) Melhor, médio e pior caso: $\Theta(n \log n)$. c) O Quick Sort clássico depende do pivô: se as partições forem extremamente desequilibradas (por exemplo, vetor já ordenado com pivô no último elemento), a recorrência vira $T(n) = T(n-1) + cn = \Theta(n^2)$. Escolher o pivô aleatoriamente reduz a probabilidade, mas não elimina o pior caso teórico. d) Se a estabilidade virar obrigatória: **Merge Sort** (estável e $\Theta(n \log n)$ no pior caso), pagando $\Theta(n)$ de memória auxiliar.

Detalhes em 08 — Heap Sort e 07 — Quick Sort.

Exercício 7 — comparando os quadráticos

`v1 = [16, 9, 28, 3, 21, 7, 35, 12, 5, 24]` (22 inversões) e `v2 = [3, 5, 7, 9, 12, 16, 21, 24, 28, 35]` (já ordenado, 0 inversões), $n = 10$.

Implementações (sem `sort()` / `sorted()`): veja o código em cada resumo — Bubble, Selection, Insertion. Para contar, basta somar 1 a um contador em cada comparação `v[j] > v[j+1]` / `v[j] < v[menor]` / `v[j] > chave` e em cada troca/deslocamento.

Algoritmo	v1: comparações	v1: trocas/deslocamentos	v2: comparações	v2: trocas/deslocamentos
Bubble sem parada antecipada	45	22 trocas	45	0
Bubble com parada (sem trocas → para)	44 (8 passagens)	22 trocas	**9** (1 passagem)	0
Selection	45	8 trocas	45	0
Insertion	29	22 deslocamentos	**9**	0

Por que esses números:

- Bubble sem parada e Selection sempre fazem $\sum_{i=1}^{n-1} i = \dfrac{n(n-1)}{2} = 45$ comparações, qualquer que seja a entrada.

- Bubble: trocas = inversões = 22. Insertion: deslocamentos = inversões = 22; comparações = deslocamentos + (inserções que pararam antes do início) = 22 + 7 = 29.
- Em `v2` (ordenado), Insertion e Bubble otimizado fazem só $n - 1 = 9$ comparações: $\Theta(n)$.

d) Em `v2`, Bubble otimizado e Insertion detectam a ordem rapidamente; Bubble sem parada e Selection continuam fazendo 45 comparações. Em `v1`, Selection faz bem menos trocas (8, no máximo $n - 1$).

e) **Mesmo pior caso, comportamentos diferentes:** os três são $\Theta(n^2)$ no pior caso (o número de comparações/inversões chega a $n(n-1)/2$), mas na entrada ordenada o Insertion e o Bubble otimizado param cedo ($\Theta(n)$), enquanto o Selection precisa procurar o menor em toda a parte restante ($\Theta(n^2)$ sempre). A notação assintótica do pior caso esconde o comportamento em outras entradas.

f) Os três são **in-place** ($\Theta(1)$ de espaço). **Estáveis:** Bubble e Insertion. **Instável:** Selection.

Exercício 8 — estabilidade na ordenação de registros

a/b) Ordenando por salário com um algoritmo estável (Insertion ou Merge Sort), mantendo a ordem original entre empates:

Salário	Funcionários (na ordem original de aparição)
2800	Carlos, Amanda
3500	Rafael, Fernanda, Juliana
4200	Marina, Bianca, Lucas
5100	Diego, Pedro

```
[("Carlos", 2800), ("Amanda", 2800), ("Rafael", 3500), ("Fernanda", 3500), ("Juliana",
3500),
 ("Marina", 4200), ("Bianca", 4200), ("Lucas", 4200), ("Diego", 5100), ("Pedro", 5100)]
```

c) Estabilidade = preservar a ordem relativa de registros com a mesma chave. Aqui, quem foi cadastrado antes continua antes entre os que ganham o mesmo salário.

d) Estáveis nas implementações estudadas: **Bubble, Insertion e Merge Sort**. Instáveis: **Selection, Quick e Heap Sort**.

e) Se os dados já estavam ordenados por outro critério (por exemplo, por nome), ordenar de forma estável pelo salário gera "salário, e dentro do mesmo salário, nome" — uma **ordenação por múltiplas chaves** sem esforço extra.

f) **Sim.** Duas formas:

- **Mudar a chave:** ordenar pelo par `(salário, posição original)`. Como não há mais empates, qualquer algoritmo produz o resultado estável (custo: memória para guardar o índice).
- **Mudar o algoritmo:** por exemplo, Selection Sort que **desloca** os elementos (em vez de trocar) ao colocar o menor na posição; ou Quick Sort com listas auxiliares (como a versão didática). Em geral custa mais tempo ou memória.

Exercício 14 — escolha do algoritmo

Cenário	Escolha	Justificativa	Alternativa e desvantagem
A: 10 elementos, quase ordenado	**Insertion Sort**	$\Theta(n)$ em entrada quase ordenada (poucas inversões), $\Theta(1)$ de memória, estável, constante pequena	Bubble otimizado: também para cedo, mas faz mais trocas; Merge Sort: overhead de recursão/memória desnecessário para 10 elementos
B: milhões de registros, preservar ordem de chaves iguais	**Merge Sort**	$\Theta(n \log n)$ no pior caso **e** estável	Heap/Quick Sort: menos memória, mas instáveis; Insertion: estável, mas $\Theta(n^2)$ é inviável para milhões
C: memória limitada, $\Theta(n \log n)$ no pior caso, espaço auxiliar constante	**Heap Sort** (heapify iterativo)	$\Theta(n \log n)$ sempre, in-place, $O(1)$	Quick Sort: in-place e rápido na média, mas pior caso $\Theta(n^2)$ e pilha até $\Theta(n)$; Merge: garantia, mas $\Theta(n)$ de memória

d) A complexidade assintótica **não** é o único critério: ela ignora constantes e termos menores (que pesam para n pequeno), memória, estabilidade, distribuição dos dados (quase ordenados?), custo relativo de comparar × mover e comportamento de cache/biblioteca.

e) No cenário B, a estabilidade faz parte da **correção** do resultado exigido: um resultado rápido, mas com a ordem dos empates trocada, está errado. Já o espaço extra $\Theta(n)$ é um custo de recurso, aceitável quando a memória disponível comporta o vetor auxiliar.

Matérias correlatas

Recursividade e recorrências — Merge, Quick e Heap Sort são analisados com as recorrências e os métodos (iteração, árvore, substituição, Mestre) de 01 — Recursividade. As somas $1 + 2 + \cdots + (n-1)$ dos quadráticos estão no apêndice desse resumo.

Pilha de recursão e in-place — espaço da pilha do Quick Sort ($\Theta(\log n)$ a $\Theta(n)$) e do `heapify` recursivo ($O(\log n)$): seção 9 de 01 — Recursividade.